

Mémo décisionnel — ThermoSense API

Groupe ThermoSense (Kenza · Enzo · Matteo · Tommy · Valentin) — Commission d'architecture, 9 juin 2026

Lecture autonome : `npm install && npm start` → <http://localhost:3000/api-docs> (compte `root / root`). Dossier complet : [15-note-5-dossier.md](#) · Résilience : [14-note-4.md](#) .

Contexte. Web service REST de supervision de capteurs et de pilotage d'actionneurs IoT pour une app mobile. Clients sur réseau **non maîtrisé** (3G d'entrepôt), devices au firmware figé. Trois décisions structurent l'architecture ; chacune est un **compromis assumé**, pas une fonctionnalité sans défaut.

Décision 1 — Idempotence côté serveur sur les écritures à effet de bord

- **Problème.** Sur réseau instable, un timeout suivi d'un retry peut **dupliquer** une mesure (moyennes et seuils d'alerte faussés) ou **rejouer** une commande d'actionneur.
- **Options écartées.** (a) Laisser le client gérer → fragile, non testable côté API. (b) `Idempotency-Key` documentée au contrat mais non honorée → c'est ce que nous avons jusqu'en S8 : protection **décorative**.
- **Choix retenu.** Header `Idempotency-Key` **UUID obligatoire** sur `POST .../mesures` et `PUT .../actuators`, **dédupliqué côté serveur** sur 24 h ([src/idempotency.js](#)) : rejeu identique → réponse mémorisée **sans ré-exécution** ; body différent → `409` ; clé absente/mal formée → `422` .
- **Compromis assumé.** Store d'idempotence **en mémoire (mono-process)** → à porter sur store partagé (Redis) en multi-instances ; la clé devient **obligatoire** (un client non conforme est refusé en `422`).

Décision 2 — Contrôle d'accès à deux étages, refus BOLA en `404`

- **Problème.** Le risque n°1 (OWASP) est le *Broken Access Control* : un utilisateur qui lit/pilote la ressource d'une autre zone, ou un device qui sort de son périmètre.
- **Choix retenu.** Autorisation **systématique en deux temps** ([src/authorization.js](#)) : d'abord **fonction** (RBAC : 4 rôles + scopes JWT → `403` si insuffisant), puis **objet** (appartenance zone/ressource → `404`). Le `404` sur violation BOLA **masque l'existence** de la ressource (anti-énumération), là où `403` resterait pour un refus de fonction.
- **Compromis assumé.** Un développeur légitime qui se trompe de zone reçoit un `404` ambigu — surcoût de diagnostic accepté pour **ne pas offrir d'oracle d'énumération** d'IDs. Couvert par **37 tests** (RBAC, BOLA, cas adverses : token expiré, signature invalide, `aud` non conforme).

Décision 3 — Résilience réseau mesurée, pas supposée (timeout + retry borné + jitter)

- **Problème.** Sans timeout applicatif, une perte de réponse **gèle** le client (≈ 8 s sans feedback) ; sans retry, les pertes transitoires deviennent des échecs.
 - **Choix retenu.** Côté client : **timeout différencié** (write 3 s / read 1,5 s) + **retry borné** (max 3, backoff exponentiel + **full jitter**). Validé par mesures reproductibles (`npm run bench`).
 - **Compromis assumé.** La résilience a un **coût** (réseau/batterie, latence de queue) et n'est **pas toujours bénéfique** : sur bon réseau elle est net négatif → elle devrait être **conditionnelle**. Le retry est rendu **sûr** par la Décision 1 (sinon il dupliquerait).
-

Risque résiduel principal (assumé)

La disponibilité sous panne réseau sévère n'est pas réglée. En perte $\geq 50\%$ (scénario C mesuré), même conforme et résilient, le succès plafonne à **66 %**, la p95 monte à **14 s** et la charge réseau **triple** (170 req/min). Le retry client **seul** ne suffit pas : il faut une **file locale + backpressure** côté client, **conçue mais non implémentée** (faute de temps). → L'**intégrité** des écritures est acquise (0 doublon) ; la **disponibilité** est le prochain chantier, clairement nommé.

Autres limites assumées : store en mémoire (données + idempotence) ; concurrence ETag/ If-Match **conçue mais non implémentée** sur PUT (last-write-wins) ; secret JWT par défaut dans le code ; écart contrat/impl sur le 429 . Détail et mitigations : [dossier §B.4 et §D.3](#).

Incident que nous allons démontrer

Doublon sur retry d'ingestion de mesure — et sa résolution serveur (S8 → S9). Scénario terrain : une passerelle en entrepôt envoie une mesure, le réseau perd la **réponse**, le client **retente**.

Pourquoi ce scénario (et pas un trivial) : il nous a *effectivement* coûté — défaut **mesuré** en S8 (14 doublons), pas supposé ; il touche l'**intégrité** des données IoT (moyennes & seuils) ; et il incarne notre fil — *mesurer avant de décider*. Il se rejoue **en direct** et se **prouve** par les chiffres.

1. **Le symptôme mesuré en S8** : à conditions identiques, la clé Idempotency-Key était **inerte** → **14 doublons** (réf. B) — preuve dans Rendu/note4/results/ (avant correctif).
2. **La démonstration live** (via le [parcours intégrateur du README](#)) : POST .../mesures avec clé → **201** ; **rejeu** même clé → **même réponse, aucun doublon** ; POST **sans** clé → **422** (enforcement).
3. **La preuve chiffrée après correctif** : rejeu du bench → **0 doublon** sur les 3 scénarios (vs 2/14/107).

Si la démo réseau live échoue, repli assumé : montrer les **logs de fallback** et les résultats `bench/` (mesures reproductibles, `npm run bench`) — le comportement dégradé est **documenté et chiffré**.